



数値計算法 第13回

数値積分 (1)

濱本 雄治¹

情報工学部 情報通信工学科

2026 年 7 月 13 日

¹hamamoto@c.oka-pu.ac.jp

復習: Newton 補間



- ▶ 標本点を追加するたびに、補間多項式に高次の項を追加

- ▷ 1 点 (x_0, y_0) を通る 0 次 Newton 補間多項式

$$p_0(x) = m_0$$

$$\text{0 次の差分商} \quad m_0 = f[x_0] \equiv y_0$$

- ▷ 2 点 $(x_0, y_0), (x_1, y_1)$ を通る 1 次 Newton 補間多項式

$$p_1(x) = m_0 + m_1(x - x_0)$$

$$\text{1 次の差分商} \quad m_1 = f[x_0, x_1] \equiv \frac{f[x_0] - f[x_1]}{x_0 - x_1}$$

- ▷ 3 点 $(x_0, y_0), (x_1, y_1), (x_2, y_2)$ を通る 2 次 Newton 補間多項式

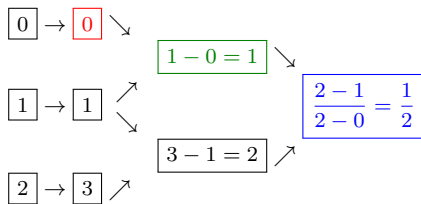
$$p_2(x) = m_0 + m_1(x - x_0) + m_2(x - x_0)(x - x_1)$$

$$\text{2 次の差分商} \quad m_2 = f[x_0, x_1, x_2] \equiv \frac{f[x_0, x_1] - f[x_1, x_2]}{x_0 - x_2}$$

前回の小テストの解説



3個の補間点 $(0, 0)$, $(1, 1)$, $(2, 3)$ を通過する関数 $y = f(x)$ について $x = 4$ における近似値を求める。差分表から2次のNewton補間多項式 $p_2(x)$ を決定し、 $f(4)$ の近似値を計算せよ。



$$\begin{aligned} p_2(x) &= m_0 + m_1(x - x_0) + m_2(x - x_0)(x - x_1) \\ &= 0 + 1 \cdot (x - 0) + \frac{1}{2}(x - 0)(x - 1) = \frac{1}{2}x^2 + \frac{1}{2}x, \\ \therefore f(4) &\simeq p_2(4) = \frac{16}{2} + \frac{4}{2} = 10 \end{aligned}$$

第13回の目標



- ▶ 定積分を数値的に実行する数値積分法を理解する
- ▶ 各種の数値積分のアルゴリズムを理解する

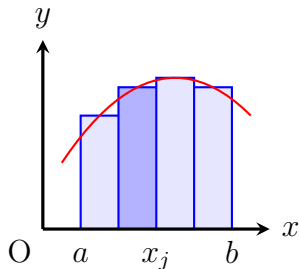
定積分



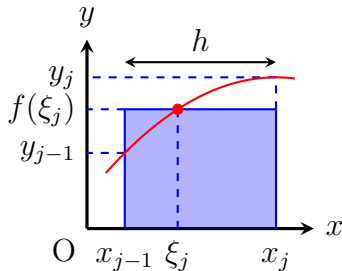
▶ ある区間を無限小区間の分割し、 $f(x)$ と x 軸が囲む面積を評価

▷ 区間 $[a, b]$ を簡単のため n 等分し、幅 $h = \frac{b-a}{n}$ を定義

$$x_j = x_0 + jh \quad (j = 0, \dots, n), \quad a = x_0 < \dots < x_n = b$$



$[x_{j-1}, x_j]$ を拡大 $\rightarrow$



▷ j 番目の長方形の高さを $f(\xi_j)$ ($x_{j-1} \leq \xi_j \leq x_j$) とすると

$$\int_a^b f(x) dx \equiv \lim_{n \rightarrow \infty} h \sum_{j=1}^n f(\xi_j)$$

($f(\xi_i)$ のとり方に依らない)

- ▶ 極限 $n \rightarrow \infty$ を取る代わりに、積分値を $n \gg 1$ の有限和で近似
 - ▷ 数値積分の基本式

$$\int_a^b f(x) dx \simeq h \sum_{j=1}^n f(\xi_j)$$

- ▷ 小区間の両端の値に $\alpha : \beta$ で重み付けした平均値で $f(\xi_j)$ を近似

$$f(\xi_j) = \frac{\alpha y_{j-1} + \beta y_j}{\alpha + \beta}$$

数値積分法の種類



- ▶ アルゴリズム毎に異なる重み α, β を採用

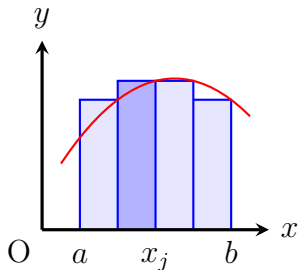
手法	α	β	長方形の高さ
区分求積法	0	1	$f(\xi_j) = y_j$
台形公式	1	1	$f(\xi_i) = \frac{y_{j-1} + y_j}{2}$
Simpson の公式 (重みは交互に変化)	1	2	$f(\xi_{j-1}) = \frac{y_{j-2} + 2y_{j-1}}{3}$
	2	1	$f(\xi_j) = \frac{2y_{j-1} + y_j}{3}$

区分求積法

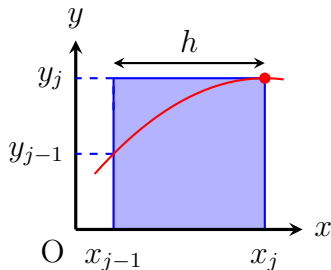


- ▶ 長方形の高さとして、右端あるいは左端の関数値を採用
 - ▷ 例えば右端の $f(\xi_j) = y_j$ を用いると

$$\int_a^b f(x) dx \simeq h \sum_{j=1}^n y_j$$



$[x_{j-1}, x_j]$ を拡大 $\rightarrow$



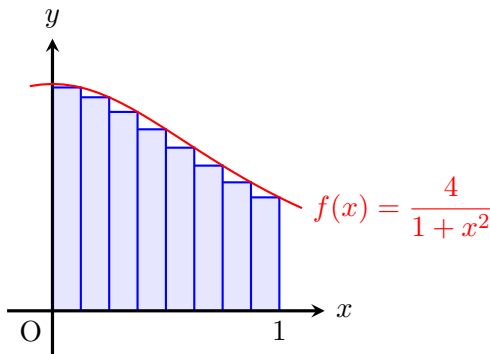
区分求積法の例



▶ 定積分

$$S = \int_0^1 \frac{4}{1+x^2} dx \quad \left(= \int_0^{\pi/4} \frac{4}{1+\tan^2 \theta} \frac{d\theta}{\cos^2 \theta} = \pi \right)$$

▷ 8分割した区分求積法の模式図



区分求積法の計算例



▶ division.c

```
#include <stdio.h>
int main(){
    double xmin = 0.0;
    double xmax = 1.0;
    for(int num=8; num<=1024; num*=2){
        double h = (xmax - xmin) / num;
        double sum = 0.0;

        for(int i=1; i<=num; ++i){
            double x = xmin + i * h;
            sum += 4.0 / (1.0 + x * x);
        }
        printf("%d %f\n", num, sum*h);
    }
}
```

▶ 実行結果

```
$ gcc division.c
$ ./a.out
8 3.013988
16 3.078442
32 3.110180
64 3.125927
128 3.133770
256 3.137684
512 3.139639
1024 3.140616
```

- ▶ 収束が遅い
- ▶ 2^{10} 分割でも精度は4桁

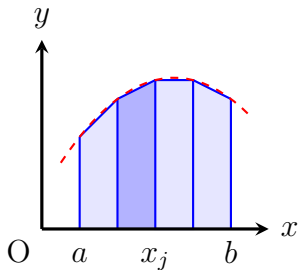
台形公式



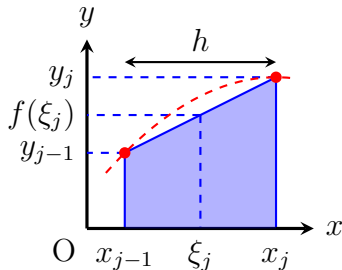
▶ 長方形の高さとして、両端の関数値の平均値を採用

▷ $f(\xi_j) = \frac{y_{j-1} + y_j}{2}$ とすると、数値積分は台形領域の面積の総和

$$\int_a^b f(x) dx \simeq h \sum_{j=1}^n \frac{y_{j-1} + y_j}{2} = h \left(\frac{y_0}{2} + \sum_{j=1}^{n-1} y_j + \frac{y_n}{2} \right) \quad (\text{台形公式})$$



$[x_{j-1}, x_j]$ を拡大 $\rightarrow$



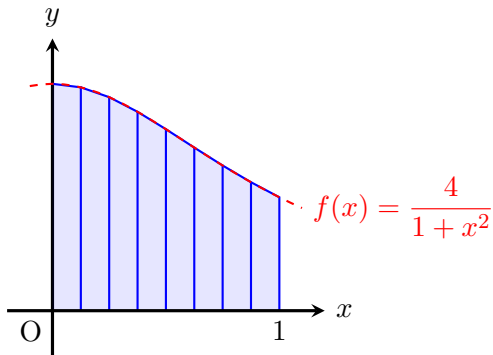
台形公式の例



▶ 定積分

$$S = \int_0^1 \frac{4}{1+x^2} dx$$

▷ 8 分割した台形公式の模式図



台形公式の計算例



▶ trapez.c

```
#include <stdio.h>
double func(double x){
    return 4.0 / (1.0 + x * x);
}
int main(){
    double xmin = 0.0; double xmax = 1.0;
    for(int num=8; num<=1024; num*=2){
        double h = (xmax - xmin) / num;
        double sum = func(xmin) + func(xmax);
        for(int i=1; i!=num; ++i){
            double x = xmin + i * h;
            sum += 2 * func(x);
        }
        printf("%d %f\n", num, 0.5 * sum * h);
    }
}
```

▶ 実行結果

```
$ gcc trapez.c
$ ../a.out
8 3.138988
16 3.140942
32 3.141430
64 3.141552
128 3.141582
256 3.141590
512 3.141592
1024 3.141592
```

▶ 区分求積法より速く
 π に収束

Simpson の公式



▶ 補間多項式の次数を上げて数値積分の精度を改善

▷ 台形公式では 2 点 $(x_{j-1}, y_{j-1}), (x_j, y_j)$ を線形補間

$$p_1(x) = y_{j-1} \frac{x - x_j}{x_{j-1} - x_j} + y_j \frac{x - x_{j-1}}{x_j - x_{j-1}} = \frac{y_j - y_{j-1}}{h} (x - x_{j-1}) + y_{j-1}$$
$$\int_{x_{j-1}}^{x_j} f_1(x) dx = h \frac{y_{j-1} + y_j}{2}$$

▷ 3 点 $(x_{j-1}, y_{j-1}), (x_j, y_j), (x_{j+1}, y_{j+1})$ を 2 次多項式で補間

$$p_2(x) = y_{j-1} \frac{(x - x_j)(x - x_{j+1})}{(x_{j-1} - x_j)(x_{j-1} - x_{j+1})} + y_j \frac{(x - x_{j-1})(x - x_{j+1})}{(x_j - x_{j-1})(x_j - x_{j+1})}$$
$$+ y_{j+1} \frac{(x - x_{j-1})(x - x_j)}{(x_{j+1} - x_{j-1})(x_{j+1} - x_j)} \quad (\text{Lagrange 補間})$$
$$= \frac{1}{2h^2} [y_{j-1}(x - x_j)(x - x_j - h) - 2y_j(x - x_j + h)(x - x_j - h)$$
$$+ y_{j+1}(x - x_j + h)(x - x_j)] \quad (\because x_{j\pm 1} = x_j \pm h)$$

Simpson の公式 (続き)



▷ $u = x - x_j$ において区間 $[x_{j-1}, x_{j+1}]$ で $f_2(x)$ を積分すると

$$\int_{x_{j-1}}^{x_{j+1}} f_2(x) dx$$

$$= \frac{1}{2h^2} \int_{-h}^h [y_{j-1}u(u-h) - 2y_j(u^2 - h^2) + y_{j+1}u(u+h)] du$$

$$= \frac{1}{h^2} \left\{ (y_{j-1} - 2y_j + y_{j+1}) \left[\frac{u^3}{3} \right]_0^h + 2y_j h^2 \left[u \right]_0^h \right\}$$

$$= h \frac{y_{j-1} + 4y_j + y_{j+1}}{3}$$

$$= h \frac{y_{j-1} + 2y_j}{3} + h \frac{2y_j + y_{j+1}}{3} \quad (\text{Simpson の公式})$$

▷ 区間 $[x_{j-1}, x_j], [x_j, x_{j+1}]$ をまとめて扱うため、**分割数 n は偶数**

合成 Simpson 公式



▶ 区間 $[a, b]$ での定積分

▷ 両端の関数値 y_0, y_n 以外の項をまとめる

$$\begin{aligned}\int_a^b f(x) dx &\simeq h \left(\frac{y_0 + 2y_1}{3} + \frac{2y_1 + y_2}{3} + \frac{y_2 + 2y_3}{3} + \frac{2y_3 + y_4}{3} \right. \\ &\quad \left. + \cdots + \frac{y_{n-2} + 2y_{n-1}}{3} + \frac{2y_{n-1} + y_n}{3} \right) \\ &= h \left[\frac{y_0}{3} + \frac{4}{3} \underbrace{(y_1 + y_3 + \cdots + y_{n-1})}_{\text{奇数番目}} \right. \\ &\quad \left. + \frac{2}{3} \underbrace{(y_2 + y_4 + \cdots + y_{n-2})}_{\text{偶数番目}} + \frac{y_n}{3} \right]\end{aligned}$$

Simpson の公式の計算例



▶ simpson.c

```
#include <stdio.h>
double func(double x){
    return 4.0 / (1.0 + x * x);
}
int main(){
    double xmin = 0.0;
    double xmax = 1.0;
    for(int num=2; num<=1024; num*=2){
        double h = (xmax - xmin) / num;
        double sum = func(xmin) + func(xmax);
        for(int j=1; j!=num; ++j){
            double x = xmin + j * h;
            sum += (j%2 == 0? 2: 4) * func(x);
        }
        printf("%d %f\n", num, sum * h / 3);
    }
}
```

▶ 実行結果

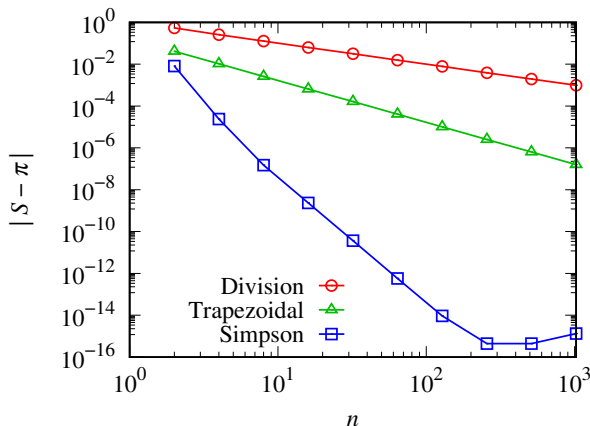
```
$ gcc simpson.c
$ ./a.out
2 3.133333
4 3.141569
8 3.141593
16 3.141593
32 3.141593
64 3.141593
128 3.141593
256 3.141593
512 3.141593
1024 3.141593
```

▶ 8 分割で 6 桁まで一致

収束性の比較



▶ $M_PI=3.14159265358979323846$ からの絶対誤差



- ▶ 分割数 n の増加とともに誤差が指数関数的に減少
- ▶ Simpson 公式では $n = 256$ で double 型の精度限界 $\sim 10^{-16}$ に到達

Newton-Cotes の公式



▶ さらに補間多項式の次数を上げる

- ▷ Simpson の公式では 2 個の区間 $[x_{j-1}, x_j], [x_j, x_{j+1}]$ をまとめて 3 点 $(x_{j-1}, y_{j-1}), (x_j, y_j), (x_{j+1}, y_{j+1})$ を 2 次関数で補間
- ▷ m 個の隣接する区間の $m + 1$ 点 $(x_0, y_0), (x_1, y_1), \dots, (x_m, y_m)$ を Lagrange 補間すると

$$f_m(x) = \sum_{i=0}^m y_i \prod_{j=0 (j \neq i)}^m \frac{x - x_j}{x_i - x_j}$$

- ▷ 区間 $[x_0, x_m]$ で $f_m(x)$ を積分すると

$$\int_{x_0}^{x_m} f_m(x) dx = \sum_{i=0}^m w_i y_i \quad (\text{Newton-Cotes の公式})$$

$$\text{ただし } w_i \equiv \int_{x_0}^{x_m} \prod_{j=0 (j \neq i)}^m \frac{x - x_j}{x_i - x_j} dx$$

第13回のまとめ



▶ 数値積分

- ▷ 定積分を分割幅が有限の和で近似

▶ 数値積分法の種類

- ▷ 区分求積法
- ▷ 台形公式: 2 点間を線形補間
- ▷ Simpson の公式: 3 点を用いて 2 次多項式で補間
- ▷ Newton-Cotes の公式: $m + 1$ 点を用いて m 次多項式で補間

▶ 注意点

- ▷ 補間多項式の次数を上げれば常に精度が上がるとは限らない
(Runge の現象)